

DocGen_Code_Analyzer_Report

DocGen — Salesforce Code Analyzer Report

AppExchange Security Review Documentation — v2.0.0 Re-submission

Package: Portwood DocGen Managed **Namespace:** portwoodglobal **Version:** 2.0.0 **Package Version Id:** 04tVx000000ZqBpIAK **Released:** Yes (promoted 2026-05-22) **Prior listing version (AppExchange):** v1.56.0 (04tal000006i1rNAAQ) — this submission addresses all 30 findings from the AppExchange security review against the v1.56 listing. v2.0 also rolls forward ~44 versions of feature work since v1.56.

Install URLs:

- **Production:** <https://login.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000ZqBpIAK>
- **Sandbox:** <https://test.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000ZqBpIAK>
- **CLI:** `sf package install --package 04tVx000000ZqBpIAK --wait 10 --target-org <your-org>`

Scan Metadata (v2.0.0)

Field	Value
Scanner	Salesforce Code Analyzer (sf code-analyzer)
Code Analyzer version	0.50.0+
Engine versions	PMD, SFGE, ESLint, RetireJS, Regex, Flow (all default versions bundled with Code Analyzer)
Scan date	2026-05-22 (against v2.0.0 source tree)
Workspace	force-app/ Security, AppExchange

Field	Value
Rule selectors	
Command	<code>sf code-analyzer run --workspace force-app/ --rule-selector Security --rule-selector AppExchange</code>
Configuration	<code>code-analyzer.yml</code> (inline suppressions enabled)

Summary — v2.0.0

Severity	Count	Disposition
1 — Critical	0	—
2 — High	0	—
3 — Moderate	38	All false positives (pattern-matching rules)
4 — Low	0	—
5 — Info	0	—
Total	38	0 exploitable findings

Additional inline-suppressed findings: 208 `code-analyzer-suppress ApexFlsViolation` markers + ~203 `NOPMD/@SuppressWarnings` markers in source, each accompanied by an inline justification comment. These mark the deliberate `WITH SYSTEM_MODE SOQL` and `AccessLevel.SYSTEM_MODE DML` sites used by the v2.0 hybrid pattern documented in § “Inline-Suppressed Findings” below.

Pass criteria: 0 High severity violations. **Status: PASS.**

Delta vs the prior submission baseline (v1.56): The v1.56 listing was scanned at 30 Moderate / 0 High on the Salesforce Code Analyzer. The v2.0 source tree is ~44 versions ahead of v1.56 (chart engine in v1.99, signature v3 placements in v1.42+, etc.) and reports 38 Moderate / 0 High. The 8-finding delta:

- **+1 `pmd:AvoidLwcBubblesComposedTrue`** (8 → 9 hits on `docGenTreeNode.js` — recursive tree node component picked up one additional event handler since v1.56 (tree-node reordering added in the post-v1.56 development cycle)).
- **+7 `pmd:ProtectSensitiveData`** (22 → 29 hits) — The signature v3 work (post-v1.56) added the `DocGen_Signature_Placement__c` object — per-tag guided signing — which contributes 4 new field metadata hits; additional fields were added to existing signature objects in the v1.57→v1.99 development cycle. All 7 new hits are documented false positives following the same pattern as the original 22.

Finding Category 1 — **pmd:AvoidLwcBubblesComposedTrue (9 Moderate, FALSE POSITIVE)**

Rule: Warns when a Lightning Web Component dispatches a CustomEvent with both `bubbles: true` and `composed: true`, because such events cross shadow DOM boundaries.

Findings: 9 — all in `lwc/docGenTreeNode/docGenTreeNode.js`.

Context: `docGenTreeNode` is a **recursive** tree component used by the V3 query-tree builder. Each node renders child nodes as additional `<c-doc-gen-tree-node>` instances. User interactions (add / remove / select / expand / reorder) must bubble from any depth back up to the root `docGenTreeBuilder` component — which lives **outside** the recursive tree's shadow DOM.

Why composed: true is required:

- Without `composed: true`, events are trapped at each node's shadow DOM boundary and never reach the tree builder.
- The alternative — chaining re-dispatch handlers at every level of the tree — would require every intermediate `docGenTreeNode` instance to listen to and re-emit every event from its children. That defeats the entire purpose of a bubbling event and makes the recursive structure unmaintainable.

Why the findings are not exploitable:

- The events only carry tree-manipulation metadata (node id, action type, field selection). They do not carry credentials, tokens, or record data.
- All event consumers are in-package LWCs (`docGenTreeBuilder`, `docGenColumnBuilder`). There is no risk of a malicious external component intercepting events, because Lightning Web Security isolates each root component instance.
- The component never renders user-supplied HTML or evaluates user-supplied strings.

Disposition: Retained intentionally. Documented in the component's JSDoc.

Finding Category 2 — **pmd:ProtectSensitiveData (29 Moderate, FALSE POSITIVE)**

Rule: PMD pattern-matches field names containing tokens such as `Token`, `Signature`, `Signer`, `Email`, `Hash`, `PIN` and flags them as “potential auth tokens with public visibility.” The rule fires on the field metadata XML file alone, without examining the field's actual sharing, CRUD, FLS, or runtime protection.

Findings: 29 field metadata files across 5 objects.

2.1 DocGen_Settings__c — 8 findings

Field	Finding rationale	Actual content & protection
Signature_Email_Brand_Color__c	Contains “Signature”	Hex color code for email branding. Not an auth token.
Signature_Email_Footer_Text__c	Contains “Signature”	Footer text for signature emails. Not an auth token.
Signature_Email_Logo_Url__c	Contains “Signature”	Relative Salesforce URL for the email logo. Not an auth token.
Signature_Email_Message__c	Contains “Signature”	Custom message included in signer emails. Not an auth token.
Signature_Email_Subject__c	Contains “Signature”	Email subject line template. Not an auth token.
Signature_OWA_Id__c	Contains “Signature”	Org-Wide Email Address Id (0D2...) used as the From address. Not a secret — it’s a Salesforce record Id already visible in Setup.
Signature_Reminder_Enabled__c	Contains “Signature”	Boolean toggle for auto-reminders. Not an auth token.
Signature_Reminder_Hours__c	Contains “Signature”	Numeric hours between reminders. Not an auth token.

DocGen_Settings__c is a **hierarchy custom setting**. Access is controlled by:

- The DocGen Admin permission set (read/write) and DocGen User permission set (read).
- Salesforce’s standard hierarchy custom setting access model.
- No guest user access — DocGen Guest Signature does not grant read on DocGen_Settings__c.

2.2 DocGen_Signature_Audit__c — 6 findings

Field	Actual content	Protection
Document_Hash_SHA256__c	SHA-256 hash of the finalized PDF (hex string).	Public by design — used for external document verification. Not a secret.
Signature_Request__c	Master-detail to DocGen_Signature_Request__c.	ControlledByParent sharing. Not a token — just a relationship field.

Field	Actual content	Protection
Signer__c	Master-detail to DocGen_Signer__c.	ControlledByParent sharing. Not a token.
Signed_Date__c	Datetime the signer completed the signing flow.	ControlledByParent sharing. Not a token.
Signer_Email__c	Email address the request was delivered to.	ControlledByParent sharing. Field history tracking enabled.
Signer_Name__c	Typed name the signer entered.	ControlledByParent sharing. Field history tracking enabled.

The DocGen_Signature_Audit__c object is **immutable by design** — it represents the legal audit record of a signing event. Write access is limited to the token-gated DocGenSignatureController.saveSignature and stampLegacySignerAndSavePdf paths; read access requires the DocGen Admin permission set OR holding the request Id / document SHA-256 hash (the public-verification capability).

2.3 DocGen_Signature_Request__c — 5 findings

Field	Actual content	Protection
Secure-Token__c	SHA-256 hex digest of the request-level token.	Token is generated fresh per request via Crypto.generateAesKey(256). Stored as a one-way hash — the plaintext cannot be recovered. Single-use. 48-hour expiry.
Signature_Data__c	Long text for capture metadata (cached preview HTML).	ControlledByParent under the parent record's sharing.
Signing_Order__c	Picklist for Sequential vs Parallel signing order.	ControlledByParent. Not a token.
Signer_Email__c	Recipient email address for the primary signer.	ControlledByParent.
Signer_Name__c	Display name for the primary signer.	ControlledByParent.

2.4 DocGen_Signer__c — 6 findings

Field	Actual content	Protection
Secure-Token__c	SHA-256 hex digest of the signer-level token.	256-bit random key → SHA-256. Plaintext never stored. Single-use. 48-hour expiry.
PIN_Hash__c	SHA-256 hash of the 6-digit email PIN.	Plaintext never stored. 10-minute expiry. 3-attempt lockout on PIN_Attempts__c.
Signature_Data__c	Typed-name SES metadata.	ControlledByParent.
Signature_Request__c	Master-detail to DocGen_Signature_Request__c.	ControlledByParent.
Signer_Email__c	Delivery address.	ControlledByParent.
Signer_Name__c	Display name.	ControlledByParent.

2.5 DocGen_Signature_Placement__c — 4 findings (new post-v1.56 — guided signing v3)

Field	Actual content	Protection
Signature_Request__c	Master-detail to DocGen_Signature_Request__c.	ControlledByParent. Not a token.
Signer__c	Master-detail to DocGen_Signer__c.	ControlledByParent. Not a token.
Signed_Value__c	The captured value for this placement (typed name, date, initials).	ControlledByParent. Not a credential.
Signed_At__c	Per-placement timestamp.	ControlledByParent. Not a credential.

Why PMD's recommended fix cannot be applied

PMD recommends: Mark the field as “Protected” or restrict visibility to a specific profile.

- Field-level “Protected” visibility does not exist as a metadata attribute for custom fields on custom objects. “Protected” visibility exists only for custom settings and custom metadata types at the **object** level in managed packages — not for individual fields.
- DocGen objects already operate under a strict permission-set model. A user without DocGen Admin, DocGen User, DocGen Guest Runner, or DocGen Guest Signature cannot read a single byte from any of these objects, because:
 - No tab grant → no app visibility.
 - No object-level read → SOQL returns zero rows.
 - No @AuraEnabled controller grant → all client calls fail with INSUFFICIENT_ACCESS.

- `Secure_Token__c` and `PIN_Hash__c` **are** the protection mechanism for the signing flow — they are cryptographic hashes, not plaintext secrets. Storing a hash is the correct pattern, not a vulnerability.
- For every other flagged field, the value is either (a) not sensitive at all (branding, color, footer text, Salesforce record Ids), or (b) protected by `ControlledByParent` sharing under the signature request, which in turn is protected by the admin permission set.

Disposition: All 29 findings are documented false positives. The fields' protection is enforced structurally, not through field-name conventions.

Inline-Suppressed Findings — v2.0.0 hybrid pattern

The v2.0.0 source ships **208 code-analyzer-suppress ApexFlsViolation markers** plus **~203 NOPMD / @SuppressWarnings markers**. Every suppression has an inline justification comment. These mark the deliberate `WITH SYSTEM_MODE SOQL` and `AccessLevel.SYSTEM_MODE DML` sites used by the v2.0 hybrid CRUD/FLS pattern:

Category	Approx. count	Location class(es)	v2.0 Justification
<code>sfge:ApexFlsViolation</code> on admin reads	~120	<code>DocGenController</code> , <code>DocGenBulkController</code> , <code>DocGenChartImageController</code> , <code>DocGenSetupController</code> , <code>DocGenSignatureSenderController</code> , <code>DocGenSignatureFlowAction</code> , <code>DocGenGiantQueryFlowAction</code> , <code>DocGenTemplateManager</code>	**Object-level <code>Schema.sObject</code> <code> isCreateable</code>
<code>sfge:ApexFlsViolation</code> on guest DML	~88	<code>DocGenSignatureController</code> , <code>DocGenAuthenticatorController</code> , <code>DocGenSignatureSenderController</code> 's preview-helper class	DocGenSignature
<code>pmd:ApexCRUDViolation</code> on bare DML	0 (eliminated in v2.0)	n/a	v2.0 eliminated

Example suppression patterns from v2.0 code:

```
// Admin path – Schema CRUD gate + SYSTEM_MODE
@AuraEnabled
public static Map<String, Object> generateDocumentData(Id
templateId, Id recordId) {
    if (templateId == null) {
        throw new DocGenException('Template ID is missing.');
```

 }

 // Explicit object-level CRUD check – the documented enforcement signal

 // gating this admin-only read on the user's DocGen permission

```

set
    // assignment.
    if (!Schema.sObjectType.DocGen_Template__c.isAccessible()) {
        throw new DocGenException('Insufficient access to DocGen
templates. Verify DocGen permission set assignment.');
```

```

    }
    // SYSTEM_MODE retained for the field read: Query_Config__c,
Header/Footer_Html__c,
    // Page_*__c are package-internal render-config long-text-area
fields.
    // USER_MODE strict FLS strips these for any admin lacking
individual FLS
    // on each field – including in managed-package build orgs
where the
    // System Administrator profile does not auto-grant FLS for
namespaced
    // custom fields. CRUD gate above is the structural
enforcement contract.
    /* code-analyzer-suppress ApexFlsViolation */
    List<DocGen_Template__c> templates = [
        SELECT Base_Object_API__c, Query_Config__c, Type__c, ...
        FROM DocGen_Template__c
        WHERE Id = :templateId
        WITH SYSTEM_MODE
        LIMIT 1
    ];
    ...
}

// Guest path – DocGenSignatureGuestSecurity helper
@AuraEnabled
@RemoteAction
public static Map<String, Object> sendPin(String token, String
email) {
    Map<String, Object> result = new Map<String, Object>();
    if (String.isBlank(token) || token.length() != 64 || !
Pattern.matches('[a-zA-F0-9]{64}', token)) {
        result.put('success', false);
        result.put('message', 'Invalid security token.');
```

```

        return result;
    }

    // Guest CRUD/FLS gate – describe check on DocGen_Signer__c
writable
    // field allowlist (PIN_Hash__c, PIN_Expires_At__c,
PIN_Attempts__c).
    // See DocGenSignatureGuestSecurity for the documented
security model.

DocGenSignatureGuestSecurity.assertSignerWritableFields(token);

    /* code-analyzer-suppress ApexFlsViolation,
DatabaseOperationsMustUseWithSharing */
    List<DocGen_Signer__c> signers = [

```



```

        SELECT Id, Signer_Email__c, PIN_Attempts__c, Status__c,
CreatedDate
        FROM DocGen_Signer__c
        WHERE Secure_Token__c = :token
        WITH SYSTEM_MODE LIMIT 1
    ];
    ...
}

```

Each code-analyzer-suppress marker is paired with a contextual comment explaining the security model at the call site. The class-level javadoc on `DocGenSignatureGuestSecurity.cls` documents the full guest security model — capability-token-bound record lookup, one-shot token rotation, PIN second factor, field allowlist enforcement.

The prior `// CxSAST: USER_MODE not viable in managed package (namespace resolution breaks unqualified field names); CRUD/FLS enforced by permission sets` rationalizations carried over from v1.56 are **gone** in v2.0 — they were demonstrably wrong (`USER_MODE` compiles fine in managed packages; the actual issue is FLS-propagation timing in package-build orgs) and the reviewer rejected the “permission set” defense. v2.0 replaces them with the Schema-CRUD-gate the reviewer’s finding language explicitly calls out as an acceptable alternative.

SFGE Engine Execution Notes

The SFGE (Salesforce Graph Engine) run may report internal execution warnings during path evaluation on the signature controllers (multi-factor validation cascades cause SFGE’s per-path timeout to trigger). These are **engine-side limitations**, not violations. SFGE traverses every possible execution path from each `@AuraEnabled` entry point; deep conditional branching in the signature controllers (token format → expiry → status → PIN verification → consent → attempts-remaining) exceeds the default 30-second per-path timeout. This is documented behavior of SFGE and does not indicate a finding.

For completeness, the same code paths are covered by:

- The Checkmarx CxSAST scan (see `DocGen_False_Positive_Report.md`).
 - 1,436 Apex tests with 76% org-wide coverage.
 - 11 end-to-end anonymous Apex scripts (`scripts/e2e-01-*.apex` through `scripts/e2e-08-*.apex` plus four `scripts/e2e-07-syntax*.apex` variants) that exercise the exact entry points SFGE timed out on.
-

Configuration — code-analyzer.yml

The repository ships `code-analyzer.yml` at the project root. It preserves all Security and AppExchange rules at their default severity and relies on inline suppression markers rather than blanket rule disabling — every suppression is documented on the line it applies to, with the justification visible in code review and audit.

No rules are downgraded. No rules are disabled globally. No paths are excluded. The 38 Moderate findings reported above are the **complete output** of the default Security + AppExchange rule set against the full `force-app/` tree.

Release Gating

Per `CLAUDE.md` — “Release Validation Checklist”:

3. Code Analyzer — Security + AppExchange (0 High violations)

```
sf code-analyzer run --workspace "force-app/" --rule-selector "Security" --rule-selector "AppExchange" --view table
```

Expected: 0 High severity violation(s) found. (38 Moderate false positives are acceptable — see `code-analyzer.yml`)

The v2.0.0 scan on 2026-05-22 meets this gate: **0 High, 38 Moderate (all documented false positives), 208 inline-suppressed ApexFlsViolation markers (all documented with v2.0 hybrid-pattern justifications).**

Cross-References

- `DocGen_Solution_Architecture_and_Usage.md` (this folder) — the security-review architecture doc.
 - `DocGen_Architecture_and_Usage.md` (this folder) — the architecture and usage companion.
 - `DocGen_False_Positive_Report.md` (this folder) — the Checkmarx CxSAST false-positive report with v2.0 hybrid-pattern rationalizations.
 - `DocGen_Platform_Technology.md` (this folder) — Salesforce platform tech inventory.
 - `../../../../SECURITY_REVIEW_RESPONSE_v2.md` — per-finding map for this re-submission.
 - `../../../../SECURITY.md` — vulnerability disclosure policy.
 - `../../../../CLAUDE.md` — release validation checklist and engineering invariants.
-